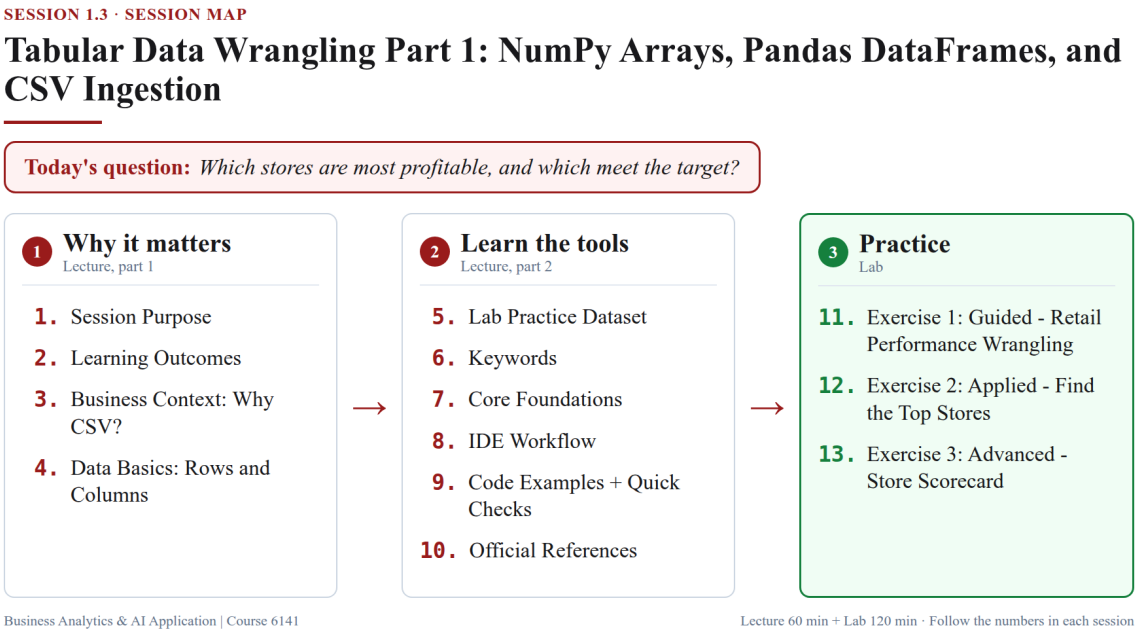
Session 1.3: Tabular Data Wrangling Part 1: NumPy Arrays, Pandas DataFrames, and CSV Ingestion

Course 6141: Business Analytics & AI Application | Reading for Week 3



Schedule: Week 3 | In-person classes (1:2 Delivery Ratio: 60 Min Lecture + 120 Min Lab)

1. Session Purpose

- Business systems (sales registers, accounting, customer databases, Excel) share tables as Comma-Separated Values (CSV) files
- Our case: a retail chain exports monthly results for its 12 stores as a CSV file
- Each row holds one store's sales, costs, visitors, and customer rating
- Your manager asks: which stores are most profitable, and which meet the target? Today, Python answers these questions for every store at once

2. Expected Learning Outcomes (By the end of this session, you will be able to)

1. "Did the store data load correctly?": read and inspect a `DataFrame` (a table in Python, like one Excel sheet)

2. “What is each store’s profit margin?”: create new columns with vectorized math (one formula runs on every row at once)
3. “Which stores meet our target?”: filter rows with `.loc[]` (keep rows that match a condition) and fix a `KeyError` (the column name does not exist)

3. Business Context: Why CSV?

- Where business data comes from: sales registers, the accounting system, the customer database, and Excel reports. Each system can export its tables as one shared file.
- `.xlsx` (Excel file) vs `.csv` (CSV file):

	`.xlsx` Excel file	`.csv` CSV file
Format	Packed format	Plain text
What you need to open it	A spreadsheet program such as Excel	Almost anything: Notepad, Excel, Google Sheets, Python `pandas`, AI assistants
Opened in Notepad	Unreadable symbols (starts with `PK`)	Readable: `Store_ID,Region_Name,Monthly_Sales_USD`

- Trade-off: CSV keeps only values, with no formulas and no formatting.

4. Data Basics: Rows and Columns

- **Row** (read across): one record. Here, all the facts about one store, such as row `1` = everything about `STR-1001`.
- **Column** (read down): one type of information for every store, such as `Monthly_Sales_USD`. In pandas, one column is a `Series`.
- **Cell:** one value where a row and a column meet, such as `185000.0`.
- Size of a table: `df.shape` returns `(12, 6)` = 12 rows, 6 columns. Row labels (the `index`) start at `0`, not 1.

5. Lab Practice Dataset

- File Link: `data/session_01_03_dataset.csv`
- Audited 12-row commercial store dataset used for vector arithmetic and filtering:
  - `Store_ID`: Unique retail location code (e.g., `STR-1001`).
  - `Region_Name`: Geographic division (East, West, North, South).
  - `Monthly_Sales_USD`: Gross store revenue in U.S. Dollars.
  - `Operating_Cost_USD`: Monthly operational store overhead in U.S. Dollars.
  - `Customer_Footfall_Count`: Total monthly foot traffic count.
  - `Customer_Satisfaction_Score`: Customer review rating (1.0 to 5.0 scale).

6. Keywords (Core Programming Concepts)

- `pandas` and `NumPy`:
  - What: free Python libraries for tables (`pandas`) and fast number arrays (`NumPy`)
  - Why: they sort, filter, and calculate tables of any size
- `CSV file` and `pd.read_csv()`:
  - What: a plain-text table where commas separate values; `pd.read_csv()` loads it
  - Why: most business systems export data as CSV
- `DataFrame`:
  - What: a whole table in Python, with row labels and column names
  - Why: it is the main object you clean, calculate, and filter
- `Series`:
  - What: one column of a DataFrame, such as `df["Monthly_Sales_USD"]`
  - Why: most calculations work column by column
- `Vectorized Computation`:
  - What: one formula applied to every row at once, with no loop
  - Why: fast, and nothing to drag down row by row
- `.loc[]` vs `.iloc[]`:
  - What: `.loc[]` selects by label or condition; `.iloc[]` selects by position number
  - Why: pull out exactly the rows a question needs
  - Example: `df.loc[df["Region_Name"] == "West"]` keeps the West rows; `df.iloc[0:2]` keeps rows 0 and 1 (`0:2` starts at 0 and stops **before** 2)

7. Core Programming & Computational Foundations

- Columns, not cells:
  - Excel: type `=(C2-D2)/C2*100` in `E2`, then drag it down; every row gets its own copy
  - Risk: someone types a number over `E5`, and that wrong row is easy to miss
  - pandas: one line covers all rows, so there is no copy to overwrite
- Table anatomy:
  - **Index:** row labels (`0`, `1`, `2` …)
  - **Columns:** header names, such as `Store_ID`
  - **Values:** the data, stored as a `NumPy` array
- Data types (`dtype`): each column has one type
  - `float64`: decimal numbers; `int64`: whole numbers; `str`: text (older pandas versions show `object`)
  - Excel: a number such as `"1,200"` saved as text is quietly skipped by `SUM`
  - pandas: that column turns into text, and `.dtypes` shows it

- Quick inspection: check before you calculate. Results for this dataset:
  - `df.shape`: `(12, 6)`, 12 rows and 6 columns
  - `df.head(2)`: the first 2 rows, `STR-1000` and `STR-1001`
  - `df.info()`: `12 non-null` in every column, so there are no gaps
  - `df.describe()`: sales mean `152416.67`, min `89000.0`, max `225000.0`

8. Development Workflow in Google Antigravity IDE

`!assets/antigravity_ide_interface.png`

- `[1]` File Explorer: create your script with your name and the session code (for example, `Reacher_S13_demo_1.py`), and check the dataset is in `data/`.
- `[2]` Code Editor: write a few lines, run, repeat. Small steps keep errors small.
- `[3]` AI Agent Chat: on an error, capture the traceback (Python's error report) with `Win + Shift + S` (macOS: `Cmd + Shift + 4`), paste it with `Ctrl + V`, and ask why it failed.
- `[4]` Terminal: run `python` followed by your script name. Return code `0` means the script finished with no errors.

9. Hands-On Coding Micro-Examples

- Every example has three parts: **1. Input** (load the data), **2. Process** (calculate or select), and **3. Output** (print the result).
- Name each file with your own name and the session code, for example `Reacher_S13_demo_1.py` (pattern: `YourName_S13_demo_1.py`).
- *Micro-Example 1 (Load a CSV File and Inspect It):* `YourName_S13_demo_1.py`

```python
# 1. Input: load the tool and the data
import pandas as pd
df = pd.read_csv("data/session_01_03_dataset.csv")

# 2. Process: measure the table and take a preview
dimensions = df.shape
column_types = df.dtypes
preview = df.head(3)

# 3. Output: show the results
print("Dataset Dimensions (Rows, Columns):", dimensions)
print("Column Data Types:\n", column_types)
print(preview)
```

  - `import pandas as pd` loads pandas under the short name `pd`. `pd.read_csv()` reads the file into a DataFrame named `df`.
  - `.shape`, `.dtypes`, and `.head(3)` give the size, the column types, and the first 3 rows. Why: confirm the data loaded correctly.
  - `print()` shows each result in the Terminal. Without it, a script shows nothing.

- *Quick Check 1 (`.tail()`):*
  - New tool: `.tail(n)` shows the last `n` rows. Why: new records are often added at the bottom.
  - Task: in part **2. Process**, change `preview = df.head(3)` to `preview = df.____(3)` so it shows the last 3 rows.
  - Check: the preview shows index `9`, `10`, `11` (`STR-1009`, `STR-1010`, `STR-1011`).

- *Micro-Example 2 (Calculate New Columns Without a Loop):* `YourName_S13_demo_2.py`

```
# 1. Input: load the store data
import pandas as pd
df = pd.read_csv("data/session_01_03_dataset.csv")

# 2. Process: profit and margin for every store at once
df["Operating_Profit_USD"] = (
    df["Monthly_Sales_USD"] - df["Operating_Cost_USD"]
)
df["Operating_Margin_Pct"] = (
    df["Operating_Profit_USD"] / df["Monthly_Sales_USD"] * 100.0
)

# 3. Output: show the first three stores
print(df[["Store_ID", "Monthly_Sales_USD",
          "Operating_Margin_Pct"]].head(3))
```

  - `df["Column_Name"]` picks one column (a `Series`). The `-`, `/`, and `*` math runs on all 12 stores at once.
  - `df[[ ... ]]` with double brackets takes a list of column names and returns a smaller table.

- *Quick Check 2 (`.round()`):*
  - New tool: `.round(n)` rounds every number in a column to `n` decimal places. Why: long decimals such as `30.985915` are hard to read in a report.
  - Task: add one line at the end of part **2. Process**: `df["Operating_Margin_Pct"] = df["Operating_Margin_Pct"].____(1)`
  - Check: the margins print as `31.0`, `34.1`, and `27.6`.

- *Micro-Example 3 (Filter Rows and Fill Missing Values):* `YourName_S13_demo_3.py`

```
# 1. Input: load the store data
import pandas as pd
df = pd.read_csv("data/session_01_03_dataset.csv")

# 2. Process: keep West stores with sales of $150,000+
west_high_sales = df.loc[
    (df["Region_Name"] == "West")
    & (df["Monthly_Sales_USD"] >= 150000.0)
]
#    then fill missing satisfaction scores with the median
```

```
median_score = df["Customer_Satisfaction_Score"].median()
df["Customer_Satisfaction_Score"] = (
    df["Customer_Satisfaction_Score"].fillna(median_score)
)

# 3. Output: show the filtered stores and the median
print(west_high_sales[["Store_ID", "Monthly_Sales_USD"]])
print("Median Satisfaction Score Imputed:", median_score)
```

  - `df.loc[condition]` keeps only the rows where the condition is `True`.
  - `&` means "and": both conditions must be true. Wrap each condition in `( )`, or Python checks them in the wrong order and fails.
  - `.fillna()` fills empty cells (`NaN`) with a value, here the median (the middle value). Today's dataset has no empty cells, so the values stay the same.

- *Quick Check 3 (`len()`):*
  - New tool: `len()` counts items; for a DataFrame, it returns the number of rows. Why: it answers "how many?" questions in one step. Example: `len(df)` returns `12`.
  - Task: add one line to part **3. Output**: `print("Stores found:", ___(west_high_sales))`
  - Check: the Terminal shows `Stores found: 3`.

10. Official Python Documentation & References

- Pandas Official Documentation: 10 Minutes to Pandas.
- Pandas Official Guide: Indexing and Selecting Data (.loc and .iloc).
- NumPy Official Documentation: NumPy Quickstart Tutorial.

11. Exercise 1: Guided Practice - Retail Performance Wrangling

- *Step 1: Open Workspace:* In Google Antigravity IDE, create a new script named `YourName_S13_wrangling.py`, replacing `YourName` with your own name (for example, `Reacher_S13_wrangling.py`). Verify that `data/session_01_03_dataset.csv` is present in Panel `[1]`.
- *Step 2: Load, Calculate, and Summarize:* Type the following Python statements:

```
# 1. Input: load the store data
import pandas as pd
df = pd.read_csv("data/session_01_03_dataset.csv")

# 2. Process: add profit and margin columns, then summarize
df["Operating_Profit_USD"] = (
    df["Monthly_Sales_USD"] - df["Operating_Cost_USD"]
)
df["Operating_Margin_Pct"] = (
    df["Operating_Profit_USD"] / df["Monthly_Sales_USD"] * 100.0
)
```

```
summary = df.describe()

# 3. Output: show summary statistics for sales and margin
print("Descriptive Financial Statistics:\n",
      summary[["Monthly_Sales_USD", "Operating_Margin_Pct"]])
```

  - `.describe()` returns summary statistics: count, mean, standard deviation, min, max, and quartiles.

- *Step 3: Compare Excel Mechanics with Pandas:*
  - In Excel, you type `=(C2-D2)/C2*100` in cell `E2` and drag it down, so every row gets its own copy of the formula. If someone types a number over `E5`, that row is wrong, and the mistake is easy to miss.
  - In Pandas, one line of `df["Operating_Margin_Pct"]` code covers every row, so there is no copy to overwrite.
- *Step 4: Execute in Terminal:* Run the script via Panel `[4]` (Terminal) using `python YourName_S13_wrangling.py`. Confirm that the summary statistics are printed.

12. Exercise 2: Applied Practice - Find the Top Stores

- *Coding Challenge:* In `YourName_S13_practice.py`, find the top stores: `Operating_Margin_Pct >= 30.0` and `Customer_Satisfaction_Score >= 4.5`.
- *Task 1 (Targeted Filtering):* Use `.loc[]` to select the qualified stores and output only `Store_ID`, `Region_Name`, and `Operating_Margin_Pct`.
- *Task 2 (Deliberate KeyError & AI Diagnosis):* Deliberately misspell a column name (e.g., `df["Monthly_Sale_USD"]` instead of `"Monthly_Sales_USD"`). Read the `KeyError` (the column name does not exist), capture the traceback with `Win + Shift + S`, paste it into Panel `[3]` (AI Agent Chat) with `Ctrl + V`, and review the AI's explanation.
- *Task 3 (Clean CSV Export):* Correct the column name, then sort with `.sort_values()`; add `ascending=False` for the highest margin first. Save the table with `.to_csv()` to `data/top_performing_stores.csv`.
- *Success Check:* the result has 5 stores, led by `STR-1011` (South, 36.89%), and the script ends with return code `0`.

13. Exercise 3: Advanced Practice - Store Scorecard

- *Goal:* Build a one-page scorecard that sorts all 12 stores into three tiers, so the regional manager can see at a glance which stores to reward and which to watch.
- *What you need:*
  - File: `data/session_01_03_dataset.csv` (12 rows, described in **5. Lab Practice Dataset**).
  - Script name: `YourName_S13_challenge.py`.
  - Tools you already know: `pd.read_csv()`, column math, `.round()`, &, `.loc[]`, `.sort_values()`, `len()`, and `.to_csv()`.
- *Requirements (follow them in order):*

1. Add `Operating_Profit_USD` = `Monthly_Sales_USD` - `Operating_Cost_USD`.
2. Add `Operating_Margin_Pct` = `Operating_Profit_USD` / `Monthly_Sales_USD` x 100, rounded to 1 decimal place.
3. Add `Sales_per_Visitor_USD` = `Monthly_Sales_USD` / `Customer_Footfall_Count`, rounded to 2 decimal places. It shows how much an average visitor spends.
4. Add a `Tier` column using these rules, in this order:
   - Every store starts as `"Standard"`.
   - `"Gold"`: `Operating_Margin_Pct >= 33.0` **and** `Customer_Satisfaction_Score >= 4.5`.
   - `"Watch"`: `Operating_Margin_Pct < 30.0`.
5. Print how many stores are in each tier, one line per tier, for example `Gold stores: 4`.
6. Print all 12 stores sorted by `Operating_Margin_Pct`, highest first, showing only `Store_ID`, `Region_Name`, `Operating_Margin_Pct`, `Sales_per_Visitor_USD`, `Customer_Satisfaction_Score`, and `Tier`.
7. Save the same sorted table to `data/store_scorecard.csv` without row labels.
8. Answer in a comment at the end of your script: which tier holds the store with the highest `Sales_per_Visitor_USD`, and what does that tell the manager?

- *New tool for this exercise: set a value for selected rows only:*
  - `df["Tier"] = "Standard"` writes one value into every row of a new column.
  - `df.loc[mask, "Tier"] = "Gold"` changes only the rows where `mask` is `True`. `mask` is a condition like the one in Micro-Example 3, with each part wrapped in `( )` and joined with `&`.
- *Hints:*
  - To count a tier, filter first, then use `len()`: `len(df.loc[df["Tier"] == "Gold"])`.
  - Pass a list of column names to `df[[ ... ]]` to keep only the columns you need.
- *Self-Check (expected results):*
  - `Gold stores: 4` (`STR-1011`, `STR-1004`, `STR-1008`, `STR-1001`), `Watch stores: 5`, `Standard stores: 3`.
  - The first row of the sorted table is `STR-1011` (South, `36.9`), and the last row is `STR-1006` (North, `23.6`).
  - The highest `Sales_per_Visitor_USD` is `STR-1002` (`11.95`), a **Watch** store with a margin of `27.6`. High spending per visitor does not guarantee a high profit margin.